

From Clarified Spec to Reviewable Plan

Turn a what-and-why spec into a reviewable how, before any code changes.

What You'll Learn

- 1 Translate a clarified spec into a technical plan
- 2 Use a plan/preview mode to review before any files change
- 3 Decompose a plan into small, independently verifiable tasks

Where Implementation Detail Belongs

The plan is where architecture choices, constraints, and non-functional requirements belong, keeping that detail out of the spec so the spec stays readable and stable.

It's the bridge between what and why (the spec) and how (the code), which is why reviewing the plan carefully matters just as much as reviewing the spec did.

Plan/Preview Mode and Task Decomposition

A reviewable plan is produced before any file changes are made: a deliberate checkpoint that exists specifically so mistakes get caught on paper, not in a diff.

The team reviews and edits the plan, including disagreeing with an architecture or scope decision, before an agent touches code, so disagreements are resolved while they're still cheap to resolve.

Break the approved plan into small, independently verifiable, agent-executable tasks. Each task should map back to a specific requirement, so it's obvious which requirement a failing task actually threatens.

A plan you didn't review is a decision you already made.

Every architecture and scope choice in the plan becomes real the moment implement runs, so silence during plan review isn't neutral, it's implicit approval of everything in the plan.

Catching a disagreement here costs a conversation. Catching it after implementation costs a rewrite, which is the entire reason this checkpoint exists before any code is touched.

APPLYING THE CONCEPTS

GitHub Spec Kit

Plan, Then Tasks

```
/speckit.plan
```

Generates design artifacts from the spec, given tech stack, architecture, and constraints as arguments, so the same spec can produce different plans depending on the stack a team actually uses.

```
/speckit.tasks
```

Generates a dependency-ordered tasks.md: Setup, Foundational, one phase per user story, and a final Polish phase. Tasks are marked for parallel execution where possible, so independent work doesn't have to wait in a single queue.

APPLYING THE CONCEPTS

OpenSpec

Design and Tasks, Together or Staged

The plan: design.md is the plan equivalent, "the how: technical approach and architecture decisions," generated alongside the proposal, so the plan and the proposal are written and reviewed as one connected pair.

The tasks: tasks.md, an implementation checklist with checkboxes, is generated in the same step by default, which keeps the plan and its breakdown from drifting apart the way a separately-written task list sometimes does.

`/opsx:continue`

Creates one artifact at a time for teams that want to review each before the next is generated, instead of the default all-at-once `/opsx:propose` or `/opsx:ff`, giving cautious teams the same staged checkpoints Spec Kit gets from separate commands.

Module 4 at a Glance

	GitHub Spec Kit	OpenSpec
Plan	/speckit.plan (separate command)	design.md (part of propose/ff)
Tasks	/speckit.tasks (separate command)	tasks.md (same step as design)
Staged review	Natural: each command is a checkpoint	/opsx:continue, one artifact at a time

Key Takeaways

- A plan you haven't reviewed is a plan you're implicitly approving, whether or not that was your intent.
- Small tasks are easier to verify and easier to isolate when something breaks, which shortens the distance from failure to fix.
- Both frameworks support staged review: Spec Kit through separate commands, OpenSpec through `/opsx:continue`, so a cautious team isn't locked out of either tool.